

Занятие 1. Расшифровка записи

Запись: GMT20260811-140013_Recording_2966x1242.mp4, 73 минуты, 2026-08-11. Ведущий: Фархад.
Формат: демонстрация экрана, три терминальных сессии Claude Code + браузер. Расшифровка отредактирована: убраны оговорки и повторы, восстановлены названия инструментов (CLAUDE.md , Claude Code, Codex, MCP, superpowers). Таймкоды — от начала файла.

Карта занятия

ТАЙМКОД	О ЧЁМ	ЧТО ПРОИСХОДИТ НА ЭКРАНЕ
01:33	Вступление	—
03:18	Ландшафт инструментов	Claude Code, Codex, VS Code
04:07	Из чего складывается доверие	—
05:50	Прогон 1: голый промпт	Папка T1, запуск <code>claude</code>
07:26	Контекстное окно	<code>/context</code>
09:51	Промпт как артефакт	Генерация промпта через Codex
11:30	Результат первого прогона	Игра 2048 в браузере
13:05	Прогон 2: настроенный репозиторий	Папка T2, <code>CLAUDE.md</code>
16:40	Правка правил руками	Победа на 64, стек → HTML
18:25	Запрет и режимы разрешений	Папка <code>Temp</code> , <code>bypass permissions</code>
23:41	Запрет сработал	Отказ удалять папку
25:17	Инициализация проекта и git	<code>git init</code> , <code>.gitignore</code>
28:51	MCP	<code>/mcp</code> , список серверов
31:20	Скиллы	Установка <code>frontend-design</code>
36:27	Хуки	—
38:14	Итог по слоям настройки	—
39:48	Скилл в действии	Переписывание интерфейса
42:18	Spec-Driven Development	Папка T3, <code>superpowers</code>
44:50	Brainstorming	Вопросы агента по объёму фич
48:11	Спецификация	Архитектура, правки человеком
49:59	Аудит независимой моделью	Вызов Codex из Claude Code
55:12	Результаты прогонов	Итоговый дизайн, игра до 64
57:43	13 находок Codex	Таблица находок на экране
59:29	Итоги	—
61:17	Вопросы и ответы	—

Вступление

01:33 Всем привет, друзья. Меня зовут Фархад, сегодня я расскажу и покажу на разных примерах, как всё это работает. Вопросы по возможности пишите в чат, я его смотрю.

02:28 История в целом всем знакомая: все уже примерно знают, что это такое и как работает. Кто-то из вас пользовался, кто-то ещё нет. Урок вводный и практический: кому-то многое будет знакомо, кто-то подчеркнёт для себя новое, для кого-то всё будет в новинку. Моё личное мнение — этим нужно пользоваться просто чтобы ускорить свою работу: программиста, тестировщика и вообще всех остальных. Сейчас есть очень много разных моделей, все они работают немного по-разному, но делают примерно одно и то же.

03:18 Мы сегодня всё будем рассматривать на примере Claude Code и модели Opus 5. Есть Claude от Anthropic — десктопное приложение выглядит примерно так. Есть Codex — выглядит примерно так. Есть разные китайские варианты, тоже очень неплохие. Есть VS Code, в который можно поставить и Claude Code, и Codex, и пользоваться ими там. Что выгодно отличает приложение от чата в браузере: нам не нужно копировать код и усложнять себе жизнь. Мы, можно сказать, запускаем агента к себе на компьютер, и он что-то у нас делает.

Из чего складывается доверие к агенту

04:07 Складывается вопрос: как мы вообще можем доверять тому, что он сделал? Доверие заслуживается временем использования конкретного агента и конкретной модели, и складывается из нескольких параметров: предсказуемость, ограниченность, компетентность, проверяемость.

То есть мы должны примерно понимать, как ведёт себя модель. Должны расставить ограничения, чтобы она не делала того, что нам не нужно. Должны дать модели инструменты, которых в ней нет, чтобы она делала именно то, что нужно.

05:00 Рассказать ей конкретно, какие у нас правила в проекте, чтобы ограничить её действия рамками того, что нас интересует. И, естественно, всё это проверять. Сейчас большая часть задачи разработчика и тестировщика сводится к тому, чтобы валидировать результаты работы ИИ — на это уходит самый большой промежуток времени. Написание кода стало сильно дешевле, поэтому сегодня мы научимся немного упрощать себе этот процесс.

Давайте начинать на примере простой игры 2048.

Главное. Написание кода подешевело — подорожала проверка. Доверие к агенту не даётся авансом, оно набирается через четыре вещи: предсказуемость, ограниченность, компетентность, проверяемость.

Прогон 1: голый промпт

05:50 Создадим тестовую папку и запустим в ней наш первый проект. Мы просто запускаем Claude в нашей папке, чтобы он получил доступ к ней как к проекту. Папка лежит на десктопе, переходим туда и запускаем. Я буду запускать из терминала — субъективно так сильно удобнее, но каждому из вас доступен полный спектр инструментов и моделей, какими вы сами захотите пользоваться. Тот же Claude может открывать папки и делать примерно то же самое через UI-приложение.

06:40 Создадим простой промпт: «напиши игру 2048» — и посмотрим, что он выдаст. У меня стоит модель Opus 5, усилие — medium.

Пока он пишет, расскажу немного про контекст, про промптинг и про то, как всё устроено. Работа с моделью — это всегда немного промпт-инжиниринг и контекст-инжиниринг.

Контекстное окно

07:26 У каждой модели есть определённое окно контекста. Это то количество данных, которое вы можете дать модели, и которое она может дать обратно. Если контекст заполняется, модель дальше не может нормально работать: она его сжимает, часть данных теряется.

08:12 Глобально: на каждую новую задачу нужно создавать новую сессию, чтобы контекст был обновлён. Потому что если контекст заполнен и в нём остались какие-то баги или старые данные, модель может использовать их при новых ответах, и это никак не поможет решить новую задачу.

08:57 У GPT-5 небольшое контекстное окно — количество действий, которые вы можете в нём сделать, ограничено. У Claude это 1 миллион токенов, сильно больше, поэтому для кодинга в нём приятнее работать: помещается гораздо больше информации, он может писать гораздо больше кода — потому что всё, что он пишет, тоже идёт в этот же контекст.

Всегда нужно думать, что входит в наш контекст, а что не входит. Покажу на примере: можно открыть `context` и посмотреть, что у нас там по умолчанию. Это системный промпт, системные инструменты — тот же `bash`, `grep`, какие-то ещё команды, использование терминала.

09:51 Дальше — файлы памяти, которые хранятся в проекте или в самом Claude Code, скиллы и все ваши сообщения в чате.

Главное. Контекст — это не ваш промпт. Это системный промпт, описания инструментов, MCP, скиллы, `CLAUDE.md`, прочитанные файлы, вывод команд и весь диалог. Ваш текст в этой сумме — единицы процентов. Отсюда правило: новая задача — новая сессия.

Промпт как отдельный артефакт

Дальше — prompt engineering. Мы написали просто «напиши игру 2048». Но мы также можем попросить тот же ChatGPT или Codex сделать нам промпт.

10:44 «Сделай промпт для реализации игры 2048» — сейчас он выдаст готовый промпт, и он будет сильно лучше того, что мы написали в одну строчку. Давайте прочитаем: у нас есть игровая логика, технические требования и результат. Скопировав его, мы получим гораздо более качественный результат по игре, чем то, что сделали до этого.

11:30 Вот у нас есть готовое приложение, запускаю — в принципе, всё работает, всё круто. Даже очень классно: модели стали очень умными, доверие к ним стало сильно больше, и необходимости писать супер-профессиональные промпты, как это было в 2024–2025 годах, уже нет.

12:18 Но что мы здесь получили? Просто какую-то готовую страничку, в создании которой мы не участвовали. Он сделал так, как считал нужным сам. Доверия у нас к нему практически нет: мы не знаем, что там под капотом, сколько это ест памяти, какие использует ресурсы и так далее.

Удалим пока эту игру и посмотрим, что он сделает с нашим сгенерированным промптом.

Главное. Рабочий результат ≠ управляемый результат. Игра запустилась, но решения о стеке, архитектуре и объёме принимала модель. Проверять нечего — эталона не было.

13:05 Думаю, результат будет немного получше, но глобально примерно тот же — потому что наши требования тоже сгенерированы Codex. Это то же самое, как если бы мы просто попросили Claude что-то придумать. Это не мы настраивали.

Прогон 2: настроенный репозиторий

13:05 Давайте вместе настроим новый репозиторий, в котором мы уже укажем все необходимые данные для разработки — например, что мы конкретно хотим сделать. Запускаем Claude уже в новой папке.

13:56 Он ничего не знает о соседней папке, потому что изолирован конкретно здесь. Но доступ к компьютеру у него, естественно, есть.

Здесь мы попросим его инициализировать `CLAUDE.md` или `AGENTS.md`. Это файл, в котором будет храниться вся информация о проекте: разрешения, требования и всё остальное. Это такой вектор для LLM, который указывает, куда нужно двигаться при разработке проекта, что можно и что нельзя. Это первый круг доверия, с которого мы начинаем, чтобы немного ограничить работу модели.

14:53 «Создай `CLAUDE.md` с параметрами» — будем указывать те параметры, которые получили в сгенерированном промпте: требования, технические требования. Результат нам пока не нужен.

Посмотрим, что тем временем сделала первая сессия. Так как промпт был сильно больше и в нём было указано, что мы делаем на React, файлов стало сильно больше. Он их все создал и потихоньку пишет по требованиям. Но опять же — он сам не сделал ни `CLAUDE.md`, ни `AGENTS.md`.

15:47 То есть он в свободном полёте и делает глобально что хочет, в рамках нашего промпта. Чтобы каждый раз не писать ограничивающие промпты, и существуют такие инструменты, как `CLAUDE.md` или `AGENTS.md` — если вы используете Codex или другие модели. `AGENTS.md` более универсальный.

Откроем, посмотрим, что там написано: игровые требования, технические требования, даже архитектуру он прописал. Естественно, всё в рамках проекта, в котором вы сейчас работаете, — это нужно валидировать вручную, перепроверять и уточнять, чтобы потом не было проблем. Потому что это основной файл, по которому агент будет принимать решения.

16:40 Например, проверка победы — у нас стоит достижение 2048. Давайте попробуем поставить 64 и скажем ему реализовать игру по этому `CLAUDE.md`. Для ускорения процесса уберём React и TypeScript и напомним просто HTML — иначе будет сильно дольше. Разделение файлов пока прокомментируем.

17:36 Команды нам тоже пока не нужны. Основное, что мы проверяем, — это условие победы: чтобы он использовал этот файл и довёл игру до 64, а не до 2048. Мы спокойно можем редактировать `CLAUDE.md`, добавлять что-то своё, и сказать ему: теперь реализуй эту игру по этому `CLAUDE.md`. На самом деле, можно не указывать, что нужно использовать `CLAUDE.md`, — он должен использовать его по умолчанию. Сейчас как раз это и проверим.

Правило-запрет и режимы разрешений

18:25 Пока он пишет игру, добавим ещё немного правил. Напишем прямо с восклицательными знаками: «не удалять Temp». Создадим папку Temp.

19:20 Если не забыли: у нас два окна. В первом он ещё пишет на React по обычному промпту из Codex, во втором реализует по нашему `CLAUDE.md` — который тоже, конечно, из Codex, но будем считать, что он наш. Давайте создадим ещё одно окно и запустим Claude в этом же проекте: то есть у нас три независимых сессии — одна в папке T1 и две в T2. Это одна и та же модель, но в разных инстансах. В T2 обе сессии смотрят в один и тот же `CLAUDE.md`.

20:12 Попросим удалить папку Temp и посмотрим, что он скажет: может он её удалить или нет.

20:59 Видим, что создаётся уже очень много файлов, хотя мы просили только `index.html` — видимо, что-то я оставил в `CLAUDE.md`. Он создал вот такую не очень красивую штуку, будем её дальше улучшать. Пока работает.

А папку он удалил — но при этом сказал, что удалять нельзя. Почему так произошло? У меня стоит режим **bypass permissions**. Это режим, когда агент делает всё, как считает нужным.

21:49 Когда стоит режим **auto**, он принимает решение сам: на важные моменты спрашивает нас, на неважные — нет. Например, на доступ к терминалу с простой командой, которая не выходит в интернет, он спрашивать не будет. Есть **ручной режим**, где мы одобряем вообще всё. Есть режим, где мы одобряем редактирование, то есть изменение уже существующих файлов. Есть **режим планирования**. И есть тот, который стоял у меня, — он пропускает всё.

22:37 Но если мы напишем «ни при каких обстоятельствах не удалять папку Temp» — сохраним, создадим папку заново и попробуем ещё раз.

23:41 Видим: строгое ограничение он уже прочитал и выполнил. О чём это говорит? Что `CLAUDE.md` обязательно нужно валидировать. Перепроверять его можно глазами — обязательно — и перепроверять можно моделью: попросить пересмотреть этот файл и подсветить узкие места.

Вообще, что нужно понять про ИИ: он сейчас очень умный и многое может делать сам. Причём те вещи, которые раньше нужно было смотреть глазами, сейчас можно отдать ему, а перепроверять уже его отчёты по этим действиям.

Главное. Запрет словами — просьба, а не механизм. Сработала пара «категоричная формулировка в `CLAUDE.md` + режим разрешений». В `bypassPermissions` не помогает никакая формулировка.

24:30 Всё это, естественно, растёт по чуть-чуть, вместе с доверием. Когда у нас ненастроенный проект и мы просто промптим, отклонения модели вправо-влево очень большие. Когда мы настраиваем папку, в которой работаем, ограничения становятся жёстче, и модель уже не гуляет — работает именно в том ключе, в котором мы её, скажем так, запрограммировали.

Инициализация проекта и работа с git

25:17 В идеале первым действием нужно инициализировать сам проект: создавать `CLAUDE.md` и всю документацию. У нас `CLAUDE.md` уже создан, поэтому просто проинициализируем репозиторий. Скажем ему: проинициализируй git-репозиторий, посмотри существующие файлы и определи, что можно добавить в `.gitignore`.

26:18 Нынешние LLM очень хорошо работают с терминалом, отлично справляются с git и с DevOps-задачами. Именно для таких задач их нужно использовать в первую очередь: разобраться с деревом ветвления в git, залить куда-то код, написать тесты на существующий код — со всем этим справляется отлично.

27:08 Но когда ты непосредственно ведёшь разработку, нужно прикладывать чуть больше своего времени и сил. Если git можно доверять больше, то в разработке нужно учитывать свой стек.

Посмотрим, что попало в `.gitignore`. Отличный `.gitignore` — но его, естественно, нужно перепроверить глазами. Он инициализировал репозиторий; отсюда же мы можем делать коммиты, мержить и создавать ветки, создавать отдельные `worktrees`, в которых будут работать отдельные агенты, и потом всё это складывать в `pull request` и одобрять самому или через агента. Потому что всё это терминальные команды, а именно с ними LLM справляется лучше всего.

27:57 Самая сложная задача для LLM в плане разработки — это, наверное, дизайн. Его нужно отсматривать глазами, и для него нужно прилагать чуть больше усилий.

Три кита: MCP, скиллы, хуки

28:51 Помимо контекста и промптинга, в таких инструментах, как Claude Code и Codex, есть MCP, скиллы и хуки. По идее, это три кита, на которых всё зиждется.

MCP

28:51 MCP — это Model Context Protocol, грубо говоря, интерфейс подключения агента к какому-либо инструменту. У вас может быть браузер, Figma, Photoshop, 3ds Max — что угодно. Для большинства таких инструментов написан MCP: это просто протокол, через который агент может дёргать определённые ручки в приложении и управлять им.

29:43 Через него мы можем сделать вещи, которые просто недоступны LLM: например, открывать браузер, ходить по страницам, заполнять формы.

30:29 Давайте посмотрим `/mcp` в нашем же проекте. Это мой настроенный Claude Code, тут разные MCP.

- **Playwright MCP** — основной, которым пользуются тестировщики и веб-разработчики. Позволяет открывать браузер: вы просто говорите агенту «открой браузер, открой такой-то сайт, перейди на страницу и заполни форму», и он всё это может сделать.
- **Stitch MCP** — дизайнерская тулза от Google, позволяет делать дизайн.
- **Pencil MCP** — аналог Figma, тоже даёт доступ к инструменту и позволяет делать дизайн.
- **Context7** — наверное, номер два по популярности после Playwright. Даёт доступ к базе документации практически по всем фреймворкам и языкам, чтобы агент не брал их из памяти. Агент мог быть обучен сильно раньше, и у него устаревшие данные — Context7 их актуализирует.

Скиллы

31:20 Скиллы — это наборы правил, которые уже существуют или которые мы можем написать сами, чтобы агент умел чем-нибудь пользоваться или делал что-то лучше.

32:21 Чтобы найти скилл или написать свой, можно попросить Claude, чтобы он его сделал. Либо просто открываем GitHub и смотрим готовые скиллы — например, скиллы на дизайн. Выбираем, что нравится, естественно, смотрим по количеству звёзд. `frontend-design` — это скилл от Anthropic, один из самых крупных, ему можно доверять. Как его установить: мы просто просим Claude Code установить скилл и даём ему ссылку.

33:08 Скилл установлен. Тем временем разработка в T2 приостановилась: у LLM появились вопросы — `CLAUDE.md` изменился в момент работы. Прошу переписать с React на чистый HTML. Он увидел, что мы изменили условие победы на 64.

34:01 Скиллы очень просто устанавливаются и существуют практически на всё. Все современные так называемые агенты — это, по идее, тоже просто скиллы. Это какой-то MD-файл, в котором описано правило поведения LLM в определённой ситуации: он их знает, использует, и его поведение оказывается в определённых рамках.

34:46 Например, мы пишем Android-приложение. LLM в целом неплохо разбирается в Android, но мы-то знаем, что не прям супер. Мы пишем свой скилл для Android, куда включаем всю накопленную базу опыта: как это всё использовать, как правильно тестировать, что он должен уметь, какой стек технологий использовать, — и сохраняем.

35:33 При следующих разработках говорим ему использовать этого агента для разработки под Android — и у нас уже будет настроенный агент, который все эти данные знает. То же самое касается любого языка программирования. Всегда лучше немного потратить времени, найти хороший скилл по своему стеку, доработать его своими знаниями — и дальше строить работу через него. В долгую это будет сильно качественнее и даст более предсказуемый результат, меньше разных багов.

Скилл — это, по идее, та компетенция, которую мы даём агенту, и из неё складывается ещё чуть больше доверия к нему.

Хуки

36:27 Хуки — это определённое поведение модели в какой-то момент. По определённому событию она может что-то сделать. Например, при создании плана — когда мы разрабатываем архитектуру или что-то ещё — мы можем поставить хук валидации через стороннюю модель. Каждый план будет проходить через этот хук: агент или просто модель не сможет пройти дальше, пока хук не проверит через дополнительную модель всё, что вы сделали.

37:19 Такие хуки удобны, чтобы автоматизировать процесс и не включаться самому в моменты, когда этого не требуется, — просто чтобы ускорить разработку. Естественно, каждый разработчик в каждой сфере выбирает это по своему усмотрению.

Итог по слоям настройки

38:14 Подытожим: чтобы доверять агенту и его работе в репозитории, нам нужно —

1. Настроить в проекте правила — всё это делается через `CLAUDE.md`.
2. Настроить MCP именно к тем инструментам, которые нужны, чтобы упростить процесс тестирования и разработки и сделать его качественнее.
3. Добавить скиллы в проект, чтобы они повысили качество написания кода или качество валидации результатов — если это тестирование.
4. Добавить хуки.

Главное. Четыре слоя настройки идут именно в этом порядке: правила → инструменты → компетенция → автоматизация. Каждый следующий имеет смысл только поверх предыдущего.

39:02 Хуки на самом деле не очень обязательный, но очень удобный инструмент. По хуку можно вызывать какое-то приложение. Например, если вы iOS-разработчик и работаете в Xcode, по хуку сборки билда мы можем поднимать симулятор и смотреть приложение в нём.

Скилл в действии и коммиты

39:48 Скилл установлен. Вот такой результат у нас получился, думаю, он будет плюс-минус финальным. Сейчас прямо в новой сессии скажем ему: перепиши результат приложения и используй скилл `frontend-design`, который мы добавили. Посмотрим, какой результат получится.

40:36 Он как раз всё дописал: React и TypeScript удалены, он перечитал `CLAUDE.md` и всё сделал. Чтобы сохранять историю изменений, думаю, никому из вас не нужно напоминать, что нужно коммитить свои изменения. Можно поставить хуки по определённым параметрам, чтобы ИИ коммитил, или просто прописать в `CLAUDE.md`, что нужно делать коммиты.

41:32 Давайте напишем «на каждый важный шаг». К сожалению, это не точная метрика, которую возьмёт на себя LLM, но в вашей работе может быть другая метрика, которую будет проще отследить. `CLAUDE.md` подтянется не сейчас, а чуть позже, потому что работа уже запущена. Как видно, он запустил скилл `frontend-design` и всё это переделает.

Spec-Driven Development

42:18 Самый простой и самый качественный сейчас паттерн работы с ИИ строится через **Spec-Driven Development**. Он включает всё, что я вкратце объяснил, и упрощает использование как для новичков, так и для опытных.

Что это такое: мы заставляем модель через хуки работать определённым образом. Изначально она пишет спецификацию, по этой спецификации пишет план, и по этому плану ведёт разработку. После того как разработка завершена, выполняется проверка всей разработки независимым агентом.

43:11 Самый простой инструмент для этого — скилл `superpowers`. Мы просто просим Claude установить скилл `superpowers`, и он работает через Spec-Driven Development. Или мы устанавливаем скилл Spec-Driven Development, Claude Code сам настраивает все хуки, и дальнейшая разработка уже идёт через SDD.

43:11 Создадим новую папку T3, перейдём в неё, снова открываем Claude. Каждый раз мы открываем Claude в нашей папке: если вы работаете в нескольких разных проектах, в каждой папке Claude будет вести себя по-своему, потому что настроен по-своему.

44:05 Здесь мы просим: «давай продумаем разработку и начнём игру 2048». Скилл `superpowers` у меня уже установлен — ставится он так же, с git. Сейчас он автоматически запустится через хук.

44:50 Он выбирает `superpowers:brainstorming`, чтобы мы определили, как будем разрабатывать: задаст мне вопросы, дальше сформирует репозиторий, запустит работу, напишет спецификацию и так далее. Выберем стиль попроще, чтобы было быстрее.

А здесь уже видно: используя скилл `frontend-design`, он переделал дизайн. Он немного отличается от классического 2048, но довольно приятный.

45:42 Опять же, это не значит, что он сделает шикарный дизайн. Естественно, его нужно немного подтюнить, чтобы он растягивался для мобилок, для десктопа и для всего остального. Наверняка здесь есть ещё какие-то проблемы, которые мы не учли, потому что это новый репозиторий. Сейчас большинство таких проблем уйдёт как раз через скилл `superpowers`.

Смотрим, какой объём фич. Давайте вместе с best score выберем. Мы можем на самом деле попросить агента задавать нам вопросы самому, и тогда будет чуть проще даже без `superpowers`, `brainstorming` или режима планирования.

46:34 Использовать ИИ как интервьюера тоже очень полезно. Но когда есть готовый инструмент, грех его не использовать. Superpowers или любой скилл по Spec-Driven Development — это сейчас, можно сказать, база в разработке через ИИ.

Выберем лёгкую анимацию. Чем выше мы выбираем усилие модели, тем дольше она работает и тем лучше думает; чем меньше — тем быстрее.

47:19 Сейчас стоит medium, чтобы работала пошутнее. Если бы мы выбрали, например, x-high, скорее всего модель додумалась бы, что нужно ещё и растягивать приложение на весь экран. Опять же, это не гарантирует результат. Именно поэтому нужно настраивать CLAUDE.md, дополнительные правила в проекте, MCP — чтобы она умела пользоваться базами документации по языкам, браузером, чтобы тестировала, когда это нужно.

48:11 И нужно настраивать хуки, чтобы всё это было быстро и удобно.

Вот он набросал архитектуру. В целом окей — скажем ему: нужно растягивать и сжимать для мобилок, и пиши спецификацию.

49:03 На этом этапе очень просто отсеять все неудобные вещи, которые он сам придумал, чтобы в спецификацию попало именно то, что нужно. Когда спецификация будет готова, мы проверим её через независимого агента. Большие спецификации проще всего проверять именно так — чтобы отсеять самые базовые проблемы, и уже потом отсматривать глазами, убеждаться, что всё хорошо, и запускать режим планирования.

Аудит независимой моделью

49:59 Давайте попросим этого агента проверить нашу вторую версию приложения через Codex. Codex — это тулза от ChatGPT, которая так же, как Claude Code, доступна из терминала, поэтому мы можем смело попросить Claude Code через терминал вызвать Codex и посмотреть, что он скажет. По идее, это и есть самый базовый аудит кода, который нужно проводить.

50:50 ИИ пишет огромное количество кода, и мы не всегда успеваем его отсматривать — если пишем полностью с ИИ большой проект, или даже просто большую фичу, или даже малую, но с большим количеством файлов и строк. Поэтому базово отсеять проблемы через стороннюю модель или через независимого агента той же модели бывает очень полезно. По факту, это сейчас тоже стандарт рынка — ускорить проверку того, что мы делаем. Потому что, по идее, только проверка сейчас и нужна.

51:42 Со временем люди начнут больше доверять моделям, модели станут лучше, и останется только утверждать результат.

Вот он вызвал отдельный Codex: здесь видно, он сам задал промпт, вызвал Codex и проверяет репозиторий в целом. Посмотрим, что у нас со спекой — она записана. Открываем: видно, что он написал очень много всего, и прочитать, валидировать, уточнить и подумать займёт какое-то время.

52:33 Поэтому базово мы также можем написать «проверь через Codex» — и он проверит спецификацию. Так как кода ещё нет, есть только спецификация, это будет довольно быстро, ему будет очень просто это сделать. И план, который мы сделаем по этой спецификации, будет сильно более качественным. Соответственно, разработка по этому плану тоже будет сильно качественнее.

То есть на каждой итерации от того, сколько сил вы вложите в её написание, зависит финальный результат. Чем меньше сил вложено в начале, тем больше проблем будет в конце.

53:24 Поэтому лучше писать и проверять спецификации дорогими моделями, запускать на максимальных усилиях. Написание спецификации не очень дорогостоящее по токенам. Проверка через другую модель тоже не дорогостоящая, потому что спецификации маленькие. То же самое с планом.

Когда у нас есть хороший план, написанный по хорошей спецификации, по нему будет написан хороший код. Нельзя жалеть на это токены. Можно немного пожалеть токены на написании непосредственно кода по хорошему плану, потому что хороший код пишут многие модели и не все из них дорогие. Но продумать этот код, продумать архитектуру, фичи проекта, экраны — неважно чего — это уже задача для более умных моделей.

54:24 Поэтому flow должен строиться именно через Spec-Driven, с акцентом на спецификацию и план с дорогими моделями.

Главное. Токены вкладываются в начало цепочки. Спека и план — дорогой моделью на максимальном усилии, они маленькие. Код по хорошему плану напишет и модель подешевле.

По этой же технологии работает и оркестрация моделей — наверняка вы слышали это модное нынче слово. Дорогая и умная модель управляет более дешёвыми, создаёт их и раздаёт им задачи. По идее, она забирает на себя часть вашей работы, потому что является частичным тестером того, что сделают модели, и агрегатором знаний, которые они получают в ходе выполнения.

Результаты прогонов

55:12 Уже две минуты идёт проверка спецификации — и это на medium. Можно представить, сколько займёт проверка через Codex всего кода: сильно дольше.

Посмотрим: дизайн закончен, вот такой получился. На этом примере наглядно видно, как один скилл — `frontend-design` — сильно изменил конечный результат. Без скилла она сама выбрала оформление, со скиллом тоже сама выбрала, но уже использовала дополнительные библиотеки, украшения, добавила анимации.

56:00 Это в основном полезно для фронтенда и для мобильщиков, потому что `frontend-design` в принципе может использоваться и в мобильной разработке. Также его можно использовать при подключении к Figma. И вообще с дизайном очень много всего — и очень много самых больших проблем тоже с дизайном.

Как видно, игра шла до 64 — он учёл все наши правила из `CLAUDE.md` и ограничил игру.

56:51 `CLAUDE.md` может быть на самом деле огромным. Сейчас он небольшой, но может достигать нереальных значений — обычно это не нужно, потому что для каждой фичи или для каждого экрана можно создать отдельные файлы с правилами, которые мы укажем в `CLAUDE.md`, и агент будет читать их автоматически. `CLAUDE.md` желательно держать чистым, чтобы его было приятно читать глазами и чтобы он не запутывал самого агента, а в нём просто указывать ссылки на дополнительные файлы в репозитории.

57:43 Codex закончил проверять спецификацию, а здесь закончил проверять наше приложение: тут он работал 6 минут, а тут 5. Видимо, чтобы проверить спецификацию, потребовались какие-то сложности. Если кода будет много, значения будут различаться прям сильно.

Он нашёл реальные отклонения — что было сломано: ввод не блокировался при оверлее победы и так далее. Мы можем попросить его исправить всё это, а можем оставить так и принять эти изменения. И дальше уже запустить в работу план по этой спецификации.

58:28 По идее, это и есть основной аудит кода, доступный через LLM, исключая наши глаза, — чтобы ускорить разработку. Мы можем вызывать отдельных агентов-профессионалов по каждому направлению, которые будут проверять. Сейчас Codex был вызван просто с промптом. Если бы мы вызвали Codex со встроенным скиллом по HTML-разработке, он, может быть, нашёл бы ещё что-то. Возможно, на таком простом проекте этого достаточно.

Итоги

59:29 Наверное, для первого раза это плюс-минус всё, потому что информации получилось очень много, её нужно переварить. Каждый блок, который я назвал, сильно расширяется индивидуально для каждого человека, потому что это просто инструмент, которым в 2026 году программисты упрощают или ускоряют свою работу. Кто-то ускоряет, кто-то просто упрощает — зависит от конкретных целей человека и целей проекта.

Если проект большой, то **CLAUDE.md** ходит вместе с репозиторием, настраивается всей командой: каждый через pull request добавляет туда свои правила, которые согласовывает тимлид или куратор репозитория, проекта, фичи, — чтобы всем было комфортно разрабатывать.

60:23 Со временем у крупных компаний — Anthropic и другие крупные команды выкладывают такое на GitHub — накапливаются очень мощные **CLAUDE.md** и файлы агентов, которыми часто делятся в open source. Их тоже очень круто использовать: это отличный опыт, который прошли люди, реально их использующие. Немного подправив под свой проект, можно получать классные результаты.

Дальше по домашнему заданию — я скину все данные через Валерию.

Вопросы и ответы

[61:17] Сколько скиллов можно использовать? Можно использовать много скиллов. Они просто будут забивать изначальный контекст. Есть скиллы, которые сразу загружаются в контекст, и есть скиллы, которые используются по запросу. Те, что по запросу, можно держать в неограниченном количестве — они просто занимают память на компьютере. А те, что загружаются сразу — как мой superpowers, который вызывается по хуку, — уже будут немного есть контекст, но не критично на хороших моделях.

[62:03] Как запустить написание проекта с новой большой спекой? Вы открываете Claude Code, Codex, любую китайскую модель, которую хотите, через тот же Claude Code или стороннее приложение. Создаёте папку проекта, инициализируете в ней репозиторий, **CLAUDE.md**, прописываете все ограничения, которые считаете верными. Потому что в конечном итоге ответственность всегда на разработчике, а ИИ — просто инструмент. Запускаете, например, Spec-Driven Development, и какой-нибудь скилл superpowers проведёт вас прямо по всему flow. Естественно, валидацию нужно попросить отдельно: попросить стороннего агента валидировать спеку или дополнительно создать на это хук.

[62:49] Темы будущих занятий? Дальше будет больше security: про песочницы для агентов, про оркестрацию агентов, про более глубокий harness, которым мы обяжем нашего агента.

Сколько будет занятий? Будет видно по результатам домашней работы — от этого будем отталкиваться.

[63:35] Как работает контекст между сессиями? Контекст привязывается к конкретному чату. Каждый новый чат — это новая сессия с агентом, и контекст привязывается именно к сессии. Поэтому

новые задачи лучше начинать в новой сессии; большие фишки, даже если это продолжение, лучше делить на несколько задач и делать в разных сессиях.

64:22 Это можно обходить как раз агентами. Агент — это независимая сессия в рамках текущей, у которой свой контекст, и поэтому он не примешивается в текущую сессию. Аккумулируя результаты всех агентов, мы можем собирать только сводку того, что нам нужно, не забывая контекст поисками по файлам и остальным мусорным выполнением задач.

[65:09] Как сбрасывается контекст? Посредством компакта — это работает в любой LLM. Если мы вылезли за размер контекста, доступный модели, и вы, например, отошли от монитора и не заметили этого, модель может начать работать чуть хуже, потому что в контекст, по её мнению, попадут не самые нужные файлы. Контекст можно компактить самому, указывая, что нужно оставить, а что нет, а можно автоматически: он возьмёт примерно 60–70% последних ответов и 20–30% первых.

65:58 Либо, если там будут очень важные моменты, он сам постарается их подсветить, но не всегда получается. Про глубокий контекст-менеджмент поговорим в будущем. Базово контекст-менеджмент — это просто новая сессия и слежение за тем, чтобы хуки, скиллы и MCP не вылезали за размер контекстного окна модели, которую вы используете.

Домашнее задание — индивидуальное или по группам? По группам, но будет вектор домашнего задания, а конкретный проект будет зависеть от вас. Жёсткой привязки, что мы делаем конкретный парсер чего-то определённого, не будет — будет вариативность.

[66:45] Будет ли текстовая выгрузка занятия, брошюра, презентация? Занятие пишется. Текстовое описание занятия скину чуть позже, презентация и брошюра тоже будут.

[67:34] Критерии оценки домашних заданий? Критерии есть, но не раскрываются. Самый базовый — насколько креативно и ново то, что вы использовали, и насколько глубоко вы погрузились в использование разных инструментов, чтобы получить конечный результат. Важен не столько сам конечный результат, сколько вовлечённость в разработку самого AI-проекта: сколько сил туда было вложено. Это видно по репозиторию — по количеству того, что там докручено, как докручено и как используется.

68:27 Вы можете написать довольно простенький сервис, но очень крутой по наполнению: разными функциями, с привязанными инструментами. Пусть даже они не очень хорошо работают.

Какую подписку выбрать — Max или Pro? Зависит от того, сколько времени у вас будет на домашнее задание. Обычная Pro-подписка закончится через небольшое количество токенов, и прям супер серьёзное что-то вы сделать не сможете. Но она обнуляется каждые пять часов: если уложитесь в три пятиминутных лимита, скорее всего, сделаете что-то интересное. Если это будет Max даже за 100 долларов, хватит вообще на всё. У меня самого Max за 200, и я не очень часто упираюсь в лимиты, хотя веду около 10 проектов одновременно.

69:59 Зависит от того, как использовать. Если всегда крутить на максимальных усилиях и делать ненужные действия, лимит закончится очень быстро. На будущих занятиях поговорим про контекст: есть оптимизаторы контекста, которые сокращают расход токенов и даже немного увеличивают скорость.

На самом деле можно делать и на Codex — он тоже очень классный, это приложение от ChatGPT. Там за 20 долларов будет немного больше лимитов за чуть более слабый функционал, но пользоваться тоже можно. Claude, конечно, посильнее в плане кодинга.

70:45 Домашнее задание будет делиться по группам — для разработки, для тестирования.

71:36 Ссылки для ДЗ и всё остальное решите со своими руководителями, через них же распространят всю информацию. Домашнее задание буду составлять сам с помощью ИИ.

73:11 Спасибо за внимание, до свидания.